How This Analysis Was Built

A Plain-English Technical Walkthrough

Purpose: Explains the tools, process, and logic behind the iMeUsWe AI Chatbot Analysis

Audience: Anyone who asks "How did you do this?"

Date: April 2026

The Big Picture — What We Did

We wanted to answer one question: **"If iMeUsWe makes its free AI astrology chatbot a paid feature, will users actually pay for it?"**

Since we can't predict the future, we did the next best thing — we looked at what's already happening with competitor apps that have paid AI astrology features. We collected thousands of real user reviews, analyzed how people feel about these apps, and specifically measured whether users are willing to pay for AI chatbot features.

Collect Reviews from competitor apps → **Clean the Data** remove noise → **Analyze Sentiment** positive or negative? → **Measure WTP** will they pay? → **Generate Report** visualize findings

> **Real-world analogy:** Imagine you want to open a paid parking lot. Before investing, you go to 6 existing paid parking lots, talk to 2,000 customers, and ask "Do you like this place?" and "Is it worth the price?" That's essentially what we did — but with app reviews instead of parking lot customers.

Step 1 — Collecting the Data (Scraping Reviews)

What we did

Tool: google-play-scraper (Python library) File: 01_scrape_reviews.py

We used a program to automatically read and download user reviews from the Google Play Store for 6 competitor astrology apps. For each app, we collected up to 500 of the most recent reviews.

Why Google Play Store?

- App reviews are where people naturally talk about pricing, features, and whether something is "worth it"
- The data is public — anyone can read these reviews on the Play Store
- No special permissions or accounts needed to collect this data
- We originally planned to use Reddit (like the Duolingo study), but Reddit now requires a formal registration process to access their data programmatically

Which apps and why?

App	Why We Picked It
Co-Star (5M+ installs)	Biggest astrology app. Uses AI for personalized readings. Charges $12 per premium report.
The Pattern (1M+ installs)	Recently added an AI chat feature. Has $9/month subscription. Users actively discuss AI + payment in reviews.
Nebula (5M+ installs)	Large user base with premium subscription. AI-powered readings.
Yodha (1M+ installs)	AI-powered Q&A format — closest to a chatbot experience. Uses per-question pricing.
Yodha Daily (500K+ installs)	Companion app to Yodha. High user satisfaction — good baseline for comparison.

easyStars AI (100K+ installs)	Has an actual AI astrologer chatbot named "Pixie" — the most similar product to what iMeUsWe is considering.

> **In simple terms:** We sent a robot to the Google Play Store to read and copy down 2,212 user reviews from these 6 apps. The robot saved everything into a spreadsheet — the review text, star rating, date, and which app it came from.

What we collected per review

Field	What it is	Example
App name	Which competitor app	"Co-Star"
Review text	What the user wrote	"This app is too expensive for AI content"
Star rating	1 to 5 stars	2
Thumbs up	How many people agreed with this review	47
Date	When the review was posted	2026-03-15

Total collected: 2,212 unique reviews across all 6 apps.

Step 2 — Cleaning the Data

What we did

Tool: Python (Pandas library) File: 02_clean_data.py

We cleaned up the raw reviews — removed junk, fixed formatting, and tagged each review based on what it talks about.

What "cleaning" means

- **Removed URLs** — links like "http://..." don't help with sentiment analysis
- **Removed special characters** — emojis, weird symbols, etc.
- **Removed very short reviews** — anything under 20 characters (like "ok" or "nice") doesn't give us useful information
- **Collapsed extra spaces** — cleaned up messy formatting

Tagging reviews

We also tagged each review based on whether it mentions topics relevant to our question:

- **AI/Payment related:** Does the review mention words like "ai", "chatbot", "premium", "subscription", "paid", "price", "expensive", "free", "worth", "money", etc.?
- 927 out of 1,938 cleaned reviews were tagged as relevant to AI or payment topics

> **In simple terms:** Imagine you recorded 2,000 customer interviews. Before analyzing them, you'd clean up the transcripts — remove background noise, fix typos, and throw out the ones where the person just said "um, yeah, it's fine." That's what this step does.

After cleaning: 1,938 usable reviews remained (274 were removed as too short or duplicates).

Step 3 — Sentiment Analysis (How Do People Feel?)

What we did

Method: Star rating + keyword analysis

File: 03_sentiment_analysis.py

We gave each review a "sentiment score" from -1.0 (very negative) to +1.0 (very positive) to measure how the user feels.

How the scoring works

We combined two signals:

1. **Star rating (the main signal — 80% of the score):**
   - 1 star → score of -1.0 (very unhappy)
   - 3 stars → score of 0.0 (neutral)
   - 5 stars → score of +1.0 (very happy)
2. **Text keywords (fine-tuning — adjusts score by up to ±0.2):**
   - Positive words: "love", "amazing", "great", "accurate", "helpful", "recommend", etc.
   - Negative words: "hate", "terrible", "scam", "waste", "annoying", "crash", "fake", etc.

> **In simple terms:** If someone gives 5 stars and writes "I love this app, it's amazing!" — they get a score close to +1.0. If someone gives 1 star and writes "terrible scam, waste of money" — they get a score close to -1.0. The star rating is the main indicator, and the words fine-tune it.

Why not use a fancy AI model for sentiment?

We initially tried using a pre-trained AI model (called "RoBERTa" from HuggingFace) to analyze sentiment. However:

- It was extremely slow on a regular computer without a GPU (would have taken hours)
- It had compatibility issues with the latest software versions
- For app reviews, the star rating is already a very strong sentiment signal — the user literally tells you how they feel on a 1-5 scale

- Our hybrid approach (stars + keywords) is faster, more reliable, and produces equally valid results for this use case

Results

Sentiment	Count	What it means
Positive (score > +0.1)	1,271 (65.6%)	User is happy with the app
Negative (score < -0.1)	609 (31.4%)	User is unhappy with the app
Neutral (-0.1 to +0.1)	58 (3.0%)	User is indifferent

Step 4 — Willingness-to-Pay Analysis (Will They Pay?)

What we did

Method: Keyword matching

File: 04_wtp_analysis.py

We scanned each review for words and phrases that indicate whether the user is willing or unwilling to pay for the app's features.

How WTP scoring works

We created two lists of keywords:

Willing to Pay (positive signal)	Not Willing to Pay (negative signal)
"worth it", "would pay", "happy to pay"	"too expensive", "not worth", "overpriced"
"good value", "fair price", "subscribe"	"rip off", "waste of money", "won't pay"
"money well spent", "premium worth"	"should be free", "cancelled subscription"
"gladly pay", "subscribed"	"never pay", "scam", "unsubscribed"

The WTP score is calculated as:

```
WTP Score = (positive keyword matches - negative keyword matches) / total keyword matches Example: A review says "this is too expensive and not worth the money" → Negative matches: "too expensive" + "not worth" = 2 → Positive matches: 0 → WTP Score = (0 - 2) / 2 = -1.0 (strongly against paying)
```

In simple terms: We're counting how many times someone says something like "I'd happily pay for this" versus "this is a rip-off." If more negative payment words appear, the score goes negative. If more positive ones appear, it goes positive. Most reviews don't mention payment at all, so they score 0 (neutral).

Results

WTP Category	Count	What it means
Willing to Pay	41 (2.1%)	User expressed willingness to pay
Not Willing to Pay	56 (2.9%)	User expressed resistance to paying
Neutral	1,841 (95.0%)	No payment-related language detected

Step 5 — Segmenting the Data (General vs AI-Specific)

What we did

`File: wtp_ai_chatbot_specific.py`

This is the most important step. We separated reviews into categories to answer two different questions.

Why we needed segmentation

Not all reviews are equally relevant. A review saying "I love the daily horoscope" tells us about astrology apps in general, but nothing about AI chatbots or payment. We needed to isolate the reviews that specifically talk about what iMeUsWe is considering.

The four segments

Segment	What it answers	How we identified it	Count
General	"Do people like astrology apps?"	Reviews that don't mention AI or payment	1,458
Payment-related	"How do people feel about paying for astrology apps?"	Reviews containing words like "pay", "premium", "subscription", "expensive", "free", "price", etc.	455
AI Chatbot-related	"How do people feel about AI chatbot features?"	Reviews containing words like "ai", "chatbot", "bot", "ai generated", "gpt", "ai reading", etc.	40

AI + Payment (both)	"Will people pay for an AI chatbot?" — THE key question	Reviews that mention BOTH AI chatbot AND payment topics	15

Why this matters: The "AI + Payment" segment (15 reviews) is the most directly relevant to iMeUsWe's decision. These are real users who have experienced a paid AI astrology chatbot and are telling us exactly what they think about it. Their WTP score was -0.33 (strongly against paying) with an average star rating of 1.73/5.

Step 6 — Creating the Report

What we did

Tool: Python (Matplotlib) + HTML/CSS | Files: 05_visualize.py + HTML report

We created two outputs: a chart image (analysis_report.png) and a full written report (the PDF you received).

- **Charts:** Generated using Matplotlib (a Python charting library) — shows sentiment distribution, WTP distribution, and per-competitor comparison
- **Written report:** Created as an HTML document with professional styling, then saved as PDF using the browser's print function

Tools & Technologies Used

Tool	What it is	What we used it for
Python	A programming language (like a calculator on steroids)	The entire analysis was written in Python — it's the "engine" that runs everything
google-play-scraper	A Python add-on for reading Play Store data	Automatically downloaded reviews from the 6 competitor apps
Pandas	A Python add-on for working with spreadsheet-like data	Cleaning, filtering, sorting, and analyzing the review data
Matplotlib	A Python add-on for creating charts	Generated the bar charts and comparison visualizations
NumPy	A Python add-on for math operations	Calculating averages, scores, and statistical operations

Kiro (AI IDE)	An AI-powered coding assistant	Helped write the code, debug issues, and generate the report
HTML/CSS	Web page formatting languages	Created the professional-looking report document

Project File Structure

All the code and data lives in a folder called `imeuswe_analysis`:

```
imeuswe_analysis/ ├── config.py ← Settings (competitor list, keywords, file paths) ├── requirements.txt ← List of Python add-ons needed ├── 01_scrape_reviews.py ← Step 1: Downloads reviews from Google Play Store ├── 02_clean_data.py ← Step 2: Cleans and tags the reviews ├── 03_sentiment_analysis.py ← Step 3: Scores each review's sentiment ├── 04_wtp_analysis.py ← Step 4: Scores willingness to pay ├── 05_visualize.py ← Step 5: Creates charts ├── wtp_ai_chatbot_specific.py ← Step 5b: AI chatbot-specific deep dive ├── wtp_report.py ← Quick summary report ├── run_pipeline.py ← Runs all steps in order (one-click) ├── data/ │ ├── raw_reddit_data.csv ← Raw scraped reviews (2,212 rows) │ ├── cleaned_data.csv ← Cleaned reviews (1,938 rows) │ ├── sentiment_results.csv ← Reviews with sentiment scores │ └── wtp_analysis.csv ← Reviews with sentiment + WTP scores └── output/ ├── analysis_report.png ← Visual chart summary └── iMeUsWe_AI_Chatbot_Analysis_Report.html ← Full written report
```

How to Reproduce This Analysis

If anyone wants to run this analysis again (e.g., with updated data), here's what they need:

1. **Install Python** (version 3.10 or newer) on your computer
2. **Open a terminal/command prompt** and navigate to the `imeuswe_analysis` folder
3. **Install the required add-ons:**

   ```
   pip install -r requirements.txt
   ```

4. **Run the full pipeline:**

```
python run_pipeline.py
```

This runs all 5 steps automatically and takes about 2-3 minutes.

5. **Check the results** in the `data/` and `output/` folders

> **In simple terms:** It's like a recipe. Install the kitchen (Python), get the ingredients (requirements.txt), and press the "cook" button (run_pipeline.py). The meal (report) appears in the output folder.

Where the Idea Came From

This analysis was modeled after a study done on **Duolingo Max** — Duolingo's AI-powered premium subscription. That study:

- Scraped 10,000 Reddit posts about Duolingo Max
- Found that users liked the AI feature but wouldn't pay $30/month for it
- Used the same pipeline: scrape → clean → sentiment analysis → WTP scoring → visualize
- The finding proved correct — Duolingo's stock corrected after the initial AI hype

We adapted the same methodology for the astrology app market, replacing Reddit with Google Play Store reviews (due to Reddit's new API restrictions) and targeting astrology-specific competitors instead of language learning apps.

What Makes This Analysis Credible

- **Real user data:** These are actual reviews from real users, not survey responses or hypothetical opinions
- **Large sample:** 1,938 cleaned reviews across 6 apps provides a solid foundation
- **Multiple signals:** We don't rely on just one metric — we combine sentiment, WTP, star ratings, and actual user quotes
- **Segmented analysis:** We clearly separate general app sentiment from AI-specific and payment-specific findings

- **Reproducible:** Anyone can run the same code and get the same results — nothing is manually manipulated
- **Proven methodology:** The same approach was validated in the Duolingo study where the prediction proved accurate

What It Doesn't Tell Us

- How iMeUsWe's own users specifically feel (this is competitor data)
- Whether a very low price point (e.g., ₹49/month) would change the equation
- How iOS users feel (we only scraped Google Play Store)
- Whether user attitudes toward AI will change over time

These gaps are acknowledged in the main report under "Limitations & Next Steps."

For questions about the methodology, contact the Product & Data Analytics Team.